

CS 316: Introduction to Deep Learning

Language Models - RNN

Dr Abdul Samad

Modeling Language

- Tokens not real values (domain is countably finite)

$$p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t | w_1, \dots, w_{t-1})$$

$$p(\text{Statistics, is, fun, .})$$

$$= p(\text{Statistics})p(\text{is} | \text{Statistics})p(\text{fun} | \text{Statistics, is})p(. | \text{Statistics, is, fun})$$

- Estimating it

$$\hat{p}(\text{is} | \text{Statistics}) = \frac{n(\text{Statistics is})}{n(\text{Statistics})}$$

Markov models

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2)P(x_3)P(x_4),$$

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2 | x_1)P(x_3 | x_2)P(x_4 | x_3),$$

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2)P(x_4 | x_2, x_3).$$

$$\hat{p}(\text{is} | \text{Statistics}) = \frac{n(\text{Statistics is})}{n(\text{Statistics})}$$

Unigram, bigram, trigram

N-grams (longer sequences of tokens)

- Need smoothing (long n-grams are infrequent)

$$\hat{p}(w) = \frac{n(w) + \epsilon_1/m}{n + \epsilon_1}$$

$$\hat{p}(w' | w) = \frac{n(w, w') + \epsilon_2 \hat{p}(w')}{n(w) + \epsilon_2}$$

$$\hat{p}(w'' | w', w) = \frac{n(w, w', w'') + \epsilon_3 \hat{p}(w', w'')}{n(w, w') + \epsilon_3}$$

Talk to your friendly Bayesian if you want to do it right!

Perplexity

3 sentences generated by the models

1. “It is raining outside”

2. “It is raining banana tree”

3. “It is raining piouw;kcj pwepoiut”

$$\exp \left(-\frac{1}{n} \sum_{t=1}^n \log P(x_t \mid x_{t-1}, \dots, x_1) \right).$$

Always predicts correct or wrong or uniform distribution?

Text Preprocessing

Tokenization

The Time Machine by H. G. Wells

- Basic Idea - map text into sequence of IDs
- **Character Encoding** (each character has one ID)
 - Small vocabulary
 - Doesn't work so well
- **Word Encoding** (each word has one ID)
 - Accurate spelling
 - Doesn't work so well
- **Byte Pair Encoding** (Goldilocks zone)
 - Frequent subsequences (like syllables)

Partitioning sequences

- Dataset takes the form of a sequence of T token indices in **corpus**.
- Partition it into subsequences, where each subsequence has n tokens (time steps)
- At the beginning of each epoch, discard the first d tokens, where $d \in [0, n]$ is uniformly sampled at random.
- $m = \left\lfloor \frac{T-d}{n} \right\rfloor$ subsequences. Denote by $\mathbf{x}_t = [x_t, \dots, x_{t+n-1}]$ the length- n subsequence starting from x_t token at time step t

Partitioning sequences

- The resulting m partitioned subsequences are $\mathbf{x}_d, \mathbf{x}_{d+n}, \dots, \mathbf{x}_{d+m(n-1)}$
- Each subsequence will be used as an input sequence into the language model.
- The target sequence for any input sequence $\mathbf{x}_t$ is $\mathbf{x}_{t+1}$ with length n .

Input sequences: the time machine by h g wells

Target sequences: the time machine by h g wells

Fig. 9.3.1 Obtaining 5 pairs of input sequences and target sequences from partitioned length-5 subsequences.

Minibatch Generation

The Time Machine by H. G. Wells

The Time Machine by H. G. Wells

The Time Machine by H. G. Wells

The Time Machine by H. G. Wells

The Time Machine by H. G. Wells

The Time Machine by H. G. Wells

Minibatch Generation

- **Sequential partitioning**
 - Pick random offset
 - Distribute sequences in sequence over mini batches
 - **Dependent samples**

RNN

$$P(x_t \mid x_{t-1}, \dots, x_1) \approx P(x_t \mid h_{t-1}),$$

h_{t-1} is a *hidden state* that stores the sequence information up to time step $t - 1$.

$$h_t = f(x_t, h_{t-1}).$$

Recurrent Neural Networks (with hidden state)

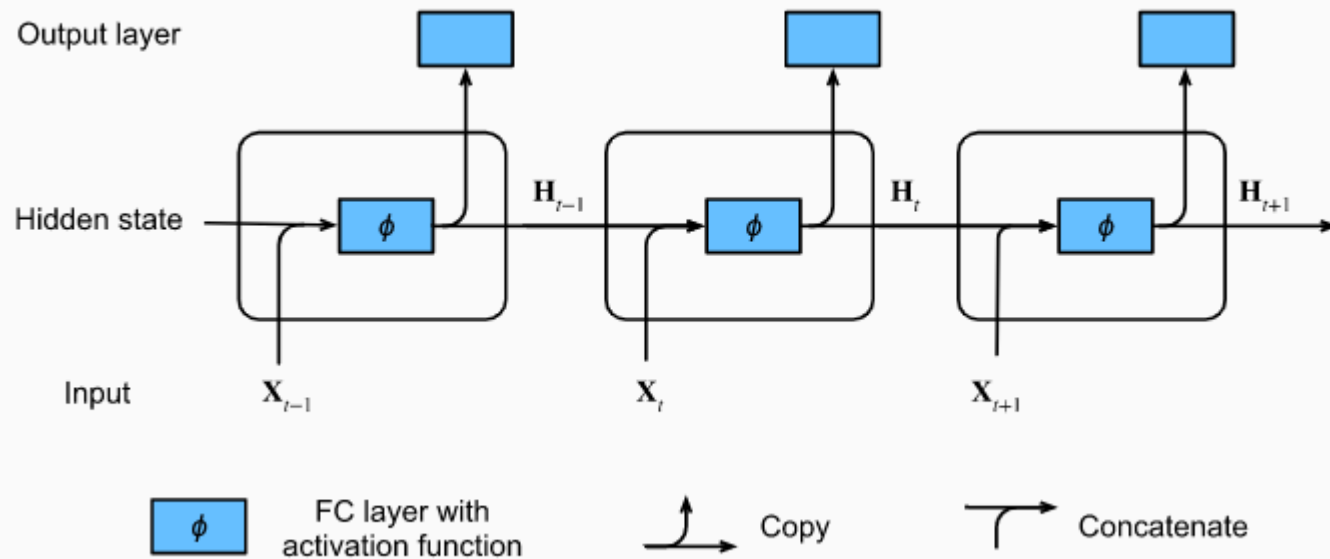
- Hidden State update

$$\mathbf{h}_t = \phi(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_{t-1} + \mathbf{b}_h)$$

- Observation update

$$\mathbf{o}_t = \phi(\mathbf{W}_{ho}\mathbf{h}_t + \mathbf{b}_o)$$

RNN



Character Level RNN

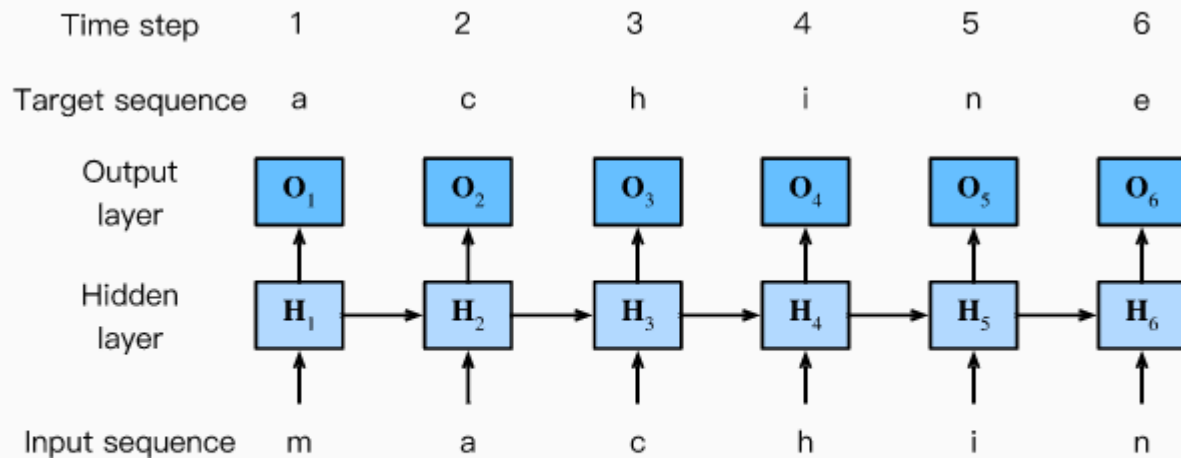


Fig. 9.4.2 A character-level language model based on the RNN. The input and target sequences are "machin" and "achine", respectively.